

PAPER 2

DATA ENGINEERING

Detailed Student Tutorial

Modules 6-9

Database & SQL • Advanced SQL & Data Modeling • Data Wrangling & Preparation • EDA & Visualization

Purpose - This tutorial explains every syllabus concept in student-friendly language, demonstrates the ideas with SQL/Python examples, and includes expected practical questions with guided solutions.

Designed for classroom learning, practical lab preparation, revision, and technical assessment practice.

How to Use This Tutorial

- Read the concept explanation first and make sure you can explain it in your own words.
- Type every SQL and Python example yourself instead of only reading the code.
- After each section, attempt the expected practical questions before checking the suggested approach.
- Use one small practice dataset throughout the modules so that SQL, preprocessing, and EDA feel connected.
- For assessment preparation, focus on explaining why a technique is used, not only its syntax.

Syllabus Coverage

Module	Main Focus	Practical Outcome
6	Database & SQL	Create databases/tables, perform CRUD, filter, sort, aggregate, retrieve ML data
7	Advanced SQL & Data Modeling	Model schemas, normalize data, join tables, write advanced analytical queries, optimize retrieval
8	Data Wrangling & Preparation	Clean, transform, integrate, validate and pipeline data for ML
9	EDA & Visualization	Profile datasets, analyze variables/relationships, visualize patterns and prepare for model building

Common Practice Dataset

The examples use a small company scenario. You can create these tables in PostgreSQL, MySQL, or another relational database with minor syntax adjustments.

managers	employees
id (PK), name, department	id (PK), name, salary, manager_id (FK)
1, Arun, IT	101, Amal, 50000, 1
2, Maya, HR	102, Riya, 62000, 1
3, Nisha, Sales	103, Nikhil, 42000, 2
	104, Fathima, 70000, 3
	105, Joel, 39000, NULL

MODULE 6 | Database & SQL

Module Goal - Understand how structured data is stored in relational databases and learn the SQL operations required to create, retrieve, modify, filter, summarize, and prepare data for machine learning.

1. Database Fundamentals

1.1 What Is a Database?

A database is an organized collection of data designed so that information can be stored, searched, updated, protected, and shared efficiently. A database is more than a file: it is normally managed by a Database Management System (DBMS), which controls storage, querying, security, concurrency, and recovery.

Example - A college database may store students, courses, attendance, marks, teachers, and fees. Instead of keeping unrelated spreadsheets, a database can connect these records through keys and relationships.

1.2 Why Databases Are Important

- Centralized storage: business information can be maintained in a controlled system.
- Fast retrieval: SQL can locate a small subset from a very large table.
- Consistency: constraints and relationships reduce conflicting or invalid data.
- Security: access can be restricted by users, roles, and permissions.
- Concurrent use: many applications or users can work with the same data safely.
- Backup and recovery: database systems support systematic recovery after failures.

1.3 Types of Databases

Type	Core Idea	Typical Use
Relational	Data stored in related tables	Business transactions, structured ML source data
Document	Records stored as documents such as JSON	Flexible application data
Key-value	Data accessed by a key	Caching, sessions, very fast lookups
Graph	Nodes and relationships are first-class concepts	Social networks, recommendation relationships
Wide-column / distributed	Large distributed datasets	High-scale data platforms

1.4 Relational Databases

A relational database stores data in tables. Each table represents an entity or concept, and tables are connected through keys. For example, the managers table stores manager details while the employees table stores employees. `employees.manager_id` can reference `managers.id`.

1.5 Database Use Cases in Machine Learning

In practical ML work, training data often originates from databases rather than ready-made CSV files. SQL is used to extract relevant rows and columns, combine tables, aggregate events, and produce a dataset that can later be cleaned and modeled in Python.

Database -> SQL Query -> Extracted Dataset -> Cleaning -> Feature Preparation -> ML Model

2. Relational Database Concepts

2.1 Tables, Rows, and Columns

A table represents one logical subject. A row is one record. A column is one attribute of that record. In an employees table, name and salary are columns, while the complete data for employee 101 is one row.

2.2 Primary Key

A primary key uniquely identifies each row. It should be unique and non-null. IDs are commonly used because names may repeat.

```
CREATE TABLE managers (
  id INT PRIMARY KEY,
  name VARCHAR(100),
  department VARCHAR(100)
);
```

2.3 Foreign Key

A foreign key creates a relationship by referencing a key in another table. It helps enforce referential integrity: an employee should not reference a manager ID that does not exist, unless null is intentionally allowed.

```
CREATE TABLE employees (
  id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  salary DECIMAL(10,2),
  manager_id INT,
  FOREIGN KEY (manager_id) REFERENCES managers(id)
);
```

2.4 Relationship Types

Relationship	Meaning	Example
One-to-One	One record relates to at most one record	Person and passport
One-to-Many	One parent can have many child records	One manager and many employees
Many-to-Many	Many records relate to many records	Students and courses; usually implemented using a junction table

3. SQL Fundamentals

3.1 SQL Syntax

SQL is a declarative language: you describe the result you want, and the database engine decides how to execute it. SQL keywords are commonly written in uppercase for readability, but many systems are case-insensitive for keywords.

```
SELECT name, salary
FROM employees
WHERE salary > 50000
ORDER BY salary DESC;
```

3.2 Creating Databases and Tables

```
CREATE DATABASE company_db;
```

After connecting to the database, create tables with clear column names, suitable data types, and constraints.

3.3 Data Types

Data Type	Stores	Example
INT	Whole numbers	employee id, age
VARCHAR(n)	Variable-length text	name, department
DECIMAL(p,s)	Exact decimal values	salary, price
BOOLEAN	True/false state	is_active
DATE	Calendar date	joined_date
TIMESTAMP	Date and time	created_at

3.4 Constraints

Constraint	Purpose
PRIMARY KEY	Uniquely identifies a row
FOREIGN KEY	Maintains a relationship between tables
NOT NULL	Prevents missing value in a required column
UNIQUE	Prevents duplicate values
CHECK	Applies a rule to allowed values
DEFAULT	Supplies a value when one is not provided

```
CREATE TABLE staff (
  id INT PRIMARY KEY,
  email VARCHAR(120) UNIQUE,
  name VARCHAR(100) NOT NULL,
  salary DECIMAL(10,2) CHECK (salary >= 0),
  is_active BOOLEAN DEFAULT TRUE
);
```

4. CRUD Operations

CRUD stands for Create, Read, Update, and Delete. These are the basic operations used by most data-driven applications.

4.1 INSERT - Create Records

```
INSERT INTO managers (id, name, department) VALUES
(1, 'Arun', 'IT'),
(2, 'Maya', 'HR'),
(3, 'Nisha', 'Sales');

INSERT INTO employees (id, name, salary, manager_id) VALUES
(101, 'Amal', 50000, 1),
(102, 'Riya', 62000, 1),
(103, 'Nikhil', 42000, 2),
(104, 'Fathima', 70000, 3),
(105, 'Joel', 39000, NULL);
```

4.2 SELECT - Read Records

```
SELECT * FROM employees;

SELECT name, salary FROM employees;
```

4.3 UPDATE - Modify Records

```
UPDATE employees
SET salary = 55000
WHERE id = 101;
```

Safety Note - Always verify the WHERE condition before running UPDATE. Without a WHERE clause, every row can be changed.

4.4 DELETE - Remove Records

```
DELETE FROM employees
WHERE id = 105;
```

Safety Note - DELETE without WHERE removes every row from the table. In real systems, test the matching rows with SELECT before executing destructive statements.

5. Filtering and Sorting

5.1 WHERE

```
SELECT * FROM employees
WHERE salary >= 50000;

SELECT * FROM employees
WHERE salary >= 50000 AND manager_id = 1;
```

Useful operators include =, <>, >, <, >=, <=, AND, OR, NOT, IN, BETWEEN, LIKE, and IS NULL.

5.2 ORDER BY

```
SELECT name, salary
FROM employees
ORDER BY salary DESC;
```

5.3 LIMIT

```
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

5.4 DISTINCT

```
SELECT DISTINCT department
FROM managers;
```

6. Aggregate Functions

Aggregate functions summarize many rows into a result. They are especially important for reporting and dataset exploration.

Function	Meaning	Example
COUNT	Number of rows	COUNT(*)
SUM	Total of numeric values	SUM(salary)
AVG	Arithmetic mean	AVG(salary)
MIN	Smallest value	MIN(salary)
MAX	Largest value	MAX(salary)

```
SELECT
    COUNT(*) AS employee_count,
    SUM(salary) AS total_salary,
```

```

    AVG(salary) AS average_salary,
    MIN(salary) AS minimum_salary,
    MAX(salary) AS maximum_salary
FROM employees;

```

Related Skill - GROUP BY is a practical companion to aggregate functions because it produces summaries per category or group.

```

SELECT manager_id, COUNT(*) AS employee_count, AVG(salary) AS avg_salary
FROM employees
GROUP BY manager_id;

```

7. ML Data Storage Concepts

7.1 Dataset Storage and Structured Data

Structured datasets have defined columns and types. Databases are useful when data changes frequently, is shared by applications, or must be queried efficiently. Extracted ML datasets may also be stored in files such as CSV or columnar formats after retrieval.

7.2 Data Retrieval Workflow

```

Operational Database -> SQL Extraction -> DataFrame -> Cleaning -> Features -> Train/Test ->
Model

```

7.3 Data Consistency

Consistency means that values follow stable rules and meanings. Examples of inconsistent values include Kerala, kerala, KERALA, and Keral representing the same category, or salary values stored with mixed numeric/text formats.

7.4 Data Quality Awareness

- Missing values
- Duplicate records
- Invalid values
- Wrong data types
- Outliers
- Inconsistent categories
- Broken relationships between tables

8. Thinking with AI for SQL

AI can explain unfamiliar SQL, propose queries, suggest schema changes, or discuss optimization. However, generated SQL must be validated against the actual schema and expected result.

- Check table and column names before running generated SQL.
- Run a SELECT version first when a generated query will UPDATE or DELETE data.
- Verify join conditions; a wrong join can silently multiply rows.
- Compare results with small manually checkable samples.
- Ask AI to explain the query so you can independently reason about correctness.

Expected Practical Questions - Module 6

1. Create managers(id, name, department) and employees(id, name, salary, manager_id) with correct primary and foreign keys.

Solution / expected approach: Define managers.id and employees.id as PRIMARY KEY. Define employees.manager_id as a FOREIGN KEY referencing managers(id). Use suitable text and numeric types.

2. Insert at least 3 managers and 5 employees.

Solution / expected approach: Use INSERT INTO ... VALUES with multiple rows. Ensure each manager_id exists or is NULL if allowed.

3. Display all employees earning more than 50,000.

Solution / expected approach: SELECT * FROM employees WHERE salary > 50000;

4. Find the top 3 highest-paid employees.

Solution / expected approach: SELECT name, salary FROM employees ORDER BY salary DESC LIMIT 3;

5. Display unique departments from managers.

Solution / expected approach: SELECT DISTINCT department FROM managers;

6. Find the total, average, minimum, and maximum salary.

Solution / expected approach: Use SUM, AVG, MIN, and MAX in one SELECT statement.

7. Count how many employees report to each manager.

Solution / expected approach: SELECT manager_id, COUNT(*) FROM employees GROUP BY manager_id;

8. Increase employee 103 salary by 10%.

Solution / expected approach: UPDATE employees SET salary = salary * 1.10 WHERE id = 103; Verify with SELECT before and after.

9. Find employees who do not currently have a manager.

Solution / expected approach: SELECT * FROM employees WHERE manager_id IS NULL;

10. Explain why UPDATE employees SET salary = 50000; is dangerous.

Solution / expected approach: There is no WHERE clause, so all employee rows will be changed.

MODULE 7 | Advanced SQL & Data Modeling

Module Goal - Design reliable relational schemas, reduce redundancy, combine multiple tables, write advanced analytical queries, use window functions, and understand performance and warehousing concepts.

1. Data Modeling

1.1 Entity Identification

An entity is a real-world object or concept that should be represented in data. For an e-commerce system, common entities include Customer, Product, Order, Order Item, and Payment. A good model starts by identifying entities before writing SQL.

1.2 Relationships and Schema Design

After identifying entities, determine how they relate. For example, one customer can place many orders, and one order can contain many products. The many-to-many relationship between orders and products is commonly resolved with an order_items junction table.

```
customers (customer_id PK, name, email)
orders (order_id PK, customer_id FK, order_date)
products (product_id PK, product_name, price)
order_items (order_id FK, product_id FK, quantity, unit_price)
```

1.3 Scalable Database Structures

- Keep one clear responsibility per table.
- Use stable keys instead of repeated descriptive text.
- Represent repeating relationships with child or junction tables.
- Choose data types that match the actual domain.
- Add indexes based on query patterns, not randomly.
- Plan for growth: query patterns that work with 100 rows may fail with millions.

2. Database Normalization

Normalization organizes data to reduce unnecessary duplication and update anomalies. The syllabus focuses on First, Second, and Third Normal Form (1NF, 2NF, 3NF).

2.1 First Normal Form (1NF)

A table is in 1NF when each cell contains one atomic value and rows are structurally consistent. A cell such as "9999, 8888" containing two phone numbers is not atomic.

Bad Design	Improved Design
customer_id=1, phones="9999,8888"	customer_phones(customer_id=1, phone=9999) and another row for 8888

2.2 Second Normal Form (2NF)

2NF requires 1NF and removes partial dependencies when a table uses a composite key. Every non-key attribute should depend on the whole composite key, not only part of it.

2.3 Third Normal Form (3NF)

3NF requires 2NF and removes transitive dependencies. A non-key attribute should not depend on another non-key attribute. If employee records contain `department_id` and `department_name`, the name should normally belong in a separate `departments` table and be referenced by `department_id`.

Why It Matters - Normalization improves consistency, reduces repeated data, and makes updates safer. It is a design technique, not a rule that every analytical table must always be maximally normalized.

3. SQL Joins

Joins combine related rows from multiple tables. The ON condition describes how rows match.

3.1 INNER JOIN

Returns only rows that match on both sides.

```
SELECT e.name AS employee, m.name AS manager, m.department
FROM employees e
INNER JOIN managers m ON e.manager_id = m.id;
```

3.2 LEFT JOIN

Returns every row from the left table and matching rows from the right. Unmatched right-side columns become NULL.

```
SELECT m.name AS manager, e.name AS employee
FROM managers m
LEFT JOIN employees e ON m.id = e.manager_id;
```

3.3 RIGHT JOIN and FULL JOIN

RIGHT JOIN preserves every row from the right table. FULL JOIN preserves rows from both sides, including unmatched rows where supported by the database system.

```
SELECT e.name, m.name
FROM employees e
FULL JOIN managers m ON e.manager_id = m.id;
```

3.4 Multi-table Queries

Real analytical queries often join several tables. Always understand the expected row grain (one row per employee, order, customer, etc.) before joining, because one-to-many joins can increase row counts.

4. Advanced Querying

4.1 Subqueries

A subquery is a query nested inside another query. It can calculate a value or return a set that the outer query uses.

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

4.2 Nested Queries

Nested queries can contain multiple levels, but excessive nesting may reduce readability. When logic becomes complex, a CTE can make steps clearer.

4.3 Common Table Expressions (CTEs)

```
WITH high_salary AS (
    SELECT id, name, salary, manager_id
    FROM employees
    WHERE salary >= 50000
)
SELECT *
FROM high_salary
ORDER BY salary DESC;
```

4.4 Complex Filtering and Analytical Queries

Advanced queries often combine joins, grouping, HAVING, subqueries, CTEs, or window functions to answer business questions rather than merely retrieve raw rows.

```
SELECT manager_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY manager_id
HAVING AVG(salary) > 50000;
```

5. Window Functions

Window functions perform calculations across related rows without collapsing the original rows. This differs from GROUP BY, which returns one row per group.

5.1 Ranking

```
SELECT name, salary,
       RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

5.2 Partitioning

```
SELECT e.name, m.department, e.salary,
       RANK() OVER (
           PARTITION BY m.department
           ORDER BY e.salary DESC
       ) AS dept_salary_rank
FROM employees e
JOIN managers m ON e.manager_id = m.id;
```

5.3 Running Totals

```
SELECT sale_date, amount,
       SUM(amount) OVER (ORDER BY sale_date) AS running_total
FROM sales;
```

5.4 Business Reporting Use Cases

- Ranking employees/products within groups
- Running revenue totals
- Comparing a row with group averages
- Period-over-period reporting
- Top-N per category

6. Database Performance

6.1 Indexes

An index is an auxiliary structure that can make lookups, filtering, joins, and ordering faster for suitable query patterns. Indexes also consume storage and add maintenance work during writes, so they should be used purposefully.

```
CREATE INDEX idx_employees_manager_id ON employees(manager_id);
CREATE INDEX idx_employees_salary ON employees(salary);
```

6.2 Query Optimization and Execution Awareness

- Select only needed columns instead of `SELECT *` when practical.
- Filter rows as early as possible when it reduces data volume.
- Join on indexed keys where appropriate.
- Avoid unnecessary repeated subqueries.
- Inspect execution plans (for example, `EXPLAIN`) when a query is slow.
- Measure performance on realistic data sizes.

```
EXPLAIN
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

7. Data Warehousing Concepts

7.1 OLTP vs OLAP

Aspect	OLTP	OLAP
Purpose	Run day-to-day transactions	Analyze data for reporting/decisions
Typical operations	Insert/update small records	Read/aggregate large datasets
Examples	Order placement, payment	Monthly sales trends, customer analysis
Data focus	Current operational data	Historical and integrated data

7.2 Data Warehouses and Analytical Databases

A data warehouse consolidates data from operational sources for analytics. It is typically designed for reporting, historical analysis, and ML feature/data pipelines rather than high-frequency transactional updates.

```
CRM + ERP + Website + Payments -> ETL/ELT -> Data Warehouse -> BI Reports / ML Datasets
```

7.3 ML Data Pipelines

ML pipelines may read from operational databases or warehouses, transform data into training features, store intermediate datasets, and deliver model-ready data. Clear data lineage helps explain where features came from and how they were produced.

8. Thinking with AI - Advanced SQL

- Ask AI to propose alternative schema designs, then compare normalization, complexity, and expected query patterns.
- Ask for multiple SQL solutions (subquery vs CTE vs window function) and reason about readability and performance.
- Validate generated joins with expected row counts.

- Use EXPLAIN or database metrics rather than accepting optimization claims without evidence.

Expected Practical Questions - Module 7

1. Design a normalized schema for students enrolling in many courses.
Solution / expected approach: Use students, courses, and enrollments(student_id, course_id, ...). The enrollments table resolves the many-to-many relationship.
2. Explain a 1NF violation where one column contains "Python, SQL, ML".
Solution / expected approach: The cell contains multiple values. Move skills into separate rows, often in a child/junction table.
3. Find employee names with their manager names.
Solution / expected approach: Use INNER JOIN employees e JOIN managers m ON e.manager_id = m.id.
4. List every manager even if they have zero employees.
Solution / expected approach: Start from managers and use LEFT JOIN employees.
5. Find employees whose salary is above the overall average.
Solution / expected approach: Use a scalar subquery: WHERE salary > (SELECT AVG(salary) FROM employees).
6. Using a CTE, first select employees with salary >= 50,000 and then order them highest to lowest.
Solution / expected approach: Create WITH high_salary AS (...) then SELECT * FROM high_salary ORDER BY salary DESC.
7. Rank employees by salary within each department.
Solution / expected approach: Join employee to manager/department and use RANK() OVER(PARTITION BY department ORDER BY salary DESC).
8. Explain GROUP BY vs window functions.
Solution / expected approach: GROUP BY collapses rows to one result per group; window functions retain row-level detail while computing across a group/window.
9. Create an index that could support queries filtering employees by manager_id.
Solution / expected approach: CREATE INDEX idx_employees_manager_id ON employees(manager_id);
10. Differentiate OLTP and OLAP with one business example each.
Solution / expected approach: OLTP: recording an order. OLAP: summarizing last year sales by region.

MODULE 8 | Data Wrangling & Preparation

Module Goal - Turn raw, inconsistent data into a reliable ML-ready dataset through cleaning, transformation, feature preparation, integration, reusable pipelines, and leakage-aware validation.

1. Data Preparation Fundamentals

1.1 Why Data Preparation Matters

Machine learning algorithms learn patterns from the data they receive. If the data contains missing values, duplicates, invalid categories, incorrect types, leakage, or inconsistent scales, model performance and trustworthiness can suffer. Preparation is therefore part of modeling, not merely a cosmetic step.

1.2 Raw vs Processed Data

Raw Value / Problem	Processed Goal
"₹50,000" stored as text	50000 as numeric
"Kerala", "kerala", "KERALA"	One standardized category
Missing age	A documented handling decision
Duplicate customer rows	One validated record when duplicate is accidental
Unscaled age vs very large income	Scaling if the chosen algorithm needs it

1.3 ML-ready Datasets

- Consistent schema and data types
- Known missing-value strategy
- Validated category values
- Features and target clearly separated
- No target/future information leakage
- Training and test transformations handled correctly
- Preparation decisions documented

1.4 Data Preparation Lifecycle

Raw Data -> Inspect -> Validate -> Clean -> Transform -> Feature Prepare -> Split -> Fit
Preprocessing -> Model

2. Data Cleaning

2.1 Missing Values

Missing data can arise from optional fields, data collection failures, unavailable measurements, or integration problems. Before filling values, inspect how much is missing and whether missingness itself has meaning.

```
df.isna().sum()

# Example strategies
df["age"] = df["age"].fillna(df["age"].median())
df = df.dropna(subset=["target"])
```

Reasoning Tip - Mean, median, mode, constant values, model-based imputation, or row removal can all be valid in different situations. The choice should be justified from data distribution and business meaning.

2.2 Duplicate Records

```
df.duplicated().sum()
df = df.drop_duplicates()
```

Do not remove duplicates automatically when repeated rows may represent legitimate repeated transactions. Identify the expected grain of the dataset first.

2.3 Invalid Entries

Examples include negative ages, impossible dates, salary stored as text, unknown category codes, or foreign keys pointing to missing entities. Invalid values should be corrected, mapped, excluded, or flagged according to a documented rule.

2.4 Outlier Identification

An outlier is an unusually distant value. Outliers can be genuine rare cases or data errors. Common detection tools include summary statistics, box plots, IQR rules, z-scores, and domain thresholds.

```
q1 = df["salary"].quantile(0.25)
q3 = df["salary"].quantile(0.75)
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
outliers = df[(df["salary"] < lower) | (df["salary"] > upper)]
```

2.5 Consistency Improvements

```
df["state"] = df["state"].str.strip().str.title()
df["email"] = df["email"].str.strip().str.lower()
```

3. Data Transformation

3.1 Feature Transformation

Transformation changes a feature into a form more useful for analysis or modeling. Examples include extracting year/month from dates, applying log transforms to skewed values, or converting annual salary into monthly salary.

3.2 Scaling and Normalization

Min-max normalization maps values to a specified range, commonly 0 to 1: $x' = (x - \min) / (\max - \min)$. This can be useful when feature scale matters.

3.3 Standardization

Standardization transforms values using $z = (x - \text{mean}) / \text{standard deviation}$, resulting in a feature centered near 0 with standard deviation near 1 on the data used to fit the transformation.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

3.4 Encoding Categorical Variables

Most ML algorithms require numerical features. Categorical values can be encoded using methods such as one-hot encoding. The encoding method should reflect whether categories are nominal or genuinely ordered.

```
encoded = pd.get_dummies(df, columns=["city"], drop_first=False)
```

4. Feature Preparation

4.1 Feature Creation

Feature creation derives new variables from existing data. Examples include `total_amount = quantity * unit_price`, `customer_age` from `date_of_birth`, or `purchase_frequency` from transaction history.

4.2 Feature Selection Awareness

Feature selection focuses the model on useful signals and can reduce noise, computation, overfitting risk, or leakage. IDs and arbitrary sequence numbers are often poor predictive features unless they encode meaningful structure.

4.3 Removing Irrelevant Information

- Pure identifiers with no predictive meaning
- Duplicated columns
- Post-outcome information that leaks target
- Free-text fields not used by the chosen pipeline
- Columns with unreliable or unusable quality

4.4 Improving Model Readiness

A model-ready feature set has consistent types, transformations fitted correctly, a clear target, and a reproducible sequence of preparation steps.

5. Data Integration

5.1 Combining and Merging Datasets

Integration combines information from multiple sources. The joining key, expected row grain, and duplicate key behavior must be understood before merging.

```
customer_orders = customers.merge(
    orders,
    on="customer_id",
    how="left"
)
```

5.2 Aggregation

```
customer_summary = orders.groupby("customer_id").agg(
    total_spend=("amount", "sum"),
    order_count=("order_id", "count"),
    avg_order_value=("amount", "mean")
).reset_index()
```

5.3 Working with Multiple Sources and Consistency

When sources use different formats, names, time zones, IDs, units, or category codes, harmonize them before combining. Integration errors can create duplicates or missing matches that look like business patterns but are actually data problems.

6. Data Pipeline Thinking

A pipeline turns one-off manual preparation into a repeatable process. Reusable transformations are critical when new training or inference data arrives.

Source -> Ingestion -> Validation -> Cleaning -> Transformation -> Features -> Storage/Model

- Keep preparation steps in code rather than undocumented manual edits.
- Fit learned transformations only on training data.
- Log assumptions and validation checks.
- Make pipeline steps testable and reusable.
- Track changes when feature logic evolves.

7. Data Leakage Awareness

7.1 What Is Data Leakage?

Data leakage occurs when information unavailable at real prediction time influences model training or evaluation. Leakage can produce unrealistically high validation scores and poor production performance.

7.2 Common Leakage Scenarios

- Using a post-outcome field as a feature (for example, `collection_completed` when predicting default before collection occurs).
- Computing preprocessing statistics on the full dataset before train/test split.
- Duplicating the same entity in both training and test sets when the task expects unseen entities.
- Using future values in a time-series feature.
- Target-derived aggregates accidentally included as predictors.

7.3 Avoiding Invalid Datasets

```
# Better pattern
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
scaler.fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```

The test set should behave like unseen future data. Any decision that learns from test values can invalidate the evaluation.

8. Thinking with AI - Data Preparation

- Ask AI to list possible quality checks from a schema, but verify them against the actual dataset.
- Ask for alternative imputation or encoding strategies and compare assumptions.
- Use AI to review a preprocessing pipeline for leakage risks.
- Do not accept transformations simply because they are common; confirm they fit the algorithm and data meaning.

Expected Practical Questions - Module 8

1. Given a DataFrame, show the missing-value count for every column.

Solution / expected approach: Use `df.isna().sum()`.

2. Fill missing age values with the median.

Solution / expected approach: `df["age"] = df["age"].fillna(df["age"].median())`.

3. Remove exact duplicate rows.

Solution / expected approach: Use `df.drop_duplicates()`, but first confirm duplicates are accidental.

4. Standardize inconsistent state values such as " kerala ", "KERALA", and "Kerala".

Solution / expected approach: Strip whitespace and apply a consistent case, e.g. `df["state"].str.strip().str.title()`.

5. Identify salary outliers using the IQR rule.

Solution / expected approach: Compute Q1, Q3, IQR; flag values below $Q1 - 1.5 \cdot IQR$ or above $Q3 + 1.5 \cdot IQR$.

6. Create total_amount from quantity and unit_price.

Solution / expected approach: `df["total_amount"] = df["quantity"] * df["unit_price"]`.

7. One-hot encode a nominal city column.

Solution / expected approach: Use `pd.get_dummies` or `OneHotEncoder`. Fit encoder using training data in an ML pipeline.

8. Merge customers with orders while preserving customers with no orders.

Solution / expected approach: Use a left merge from customers to orders on `customer_id`.

9. Create customer-level total spend and order count.

Solution / expected approach: Group orders by `customer_id` and aggregate `sum(amount)` and `count(order_id)`.

10. Explain why scaling the full dataset before splitting can be leakage.

Solution / expected approach: The scaler learns test-set statistics, allowing information from evaluation data to influence training preprocessing.

11. Create a reusable preprocessing workflow for numeric and categorical features.

Solution / expected approach: Separate numeric/categorical columns; define imputers/scalers/encoders; combine in a pipeline/column transformer; fit only on training data.

12. Document a cleaning decision for invalid negative age values.

Solution / expected approach: State the detection rule, business assumption, action taken (correct/remove/set missing), and number of affected rows.

MODULE 9 | EDA & Visualization

Module Goal - Use systematic exploratory analysis and visualization to understand datasets, identify patterns and anomalies, examine relationships, validate quality, and prepare evidence-based decisions for model building.

1. Exploratory Data Analysis (EDA)

1.1 Purpose of EDA

EDA is the structured investigation of a dataset before and during modeling. It helps answer what the data contains, how variables are distributed, which values are missing or unusual, how features relate to one another, and what issues could affect a model.

1.2 Understanding a Dataset

```
df.head()
df.shape
df.columns
df.info()
df.describe(include="all")
```

Start with structure before plotting. Identify the unit represented by one row, the target (if any), column types, missingness, and whether values are plausible.

1.3 Identifying Patterns and Anomalies

Patterns may include trends, clusters, seasonal behavior, category differences, or relationships between features. Anomalies may represent fraud, rare events, bad sensors, incorrect entry, or legitimate extreme behavior. EDA should distinguish these possibilities rather than automatically delete unusual values.

1.4 Data Storytelling

Data storytelling converts analysis into a clear message supported by evidence. A useful finding identifies what happened, the supporting statistic/visual, why it matters, and what should be investigated or done next.

2. Descriptive Analysis

2.1 Summary Statistics

Summary statistics provide a compact view of numerical data: count, mean, standard deviation, minimum, quartiles, median, and maximum. Categorical features can be summarized using frequency counts and number of unique values.

2.2 Central Tendency

Measure	Meaning	Important Note
Mean	Arithmetic average	Sensitive to extreme values
Median	Middle value after sorting	More robust to outliers
Mode	Most frequent value	Useful for categorical/discrete data

2.3 Variability

Measure	Interpretation
Range	Max - min; simple but highly sensitive to extremes
Variance	Average squared deviation concept
Standard deviation	Typical spread around the mean, in original unit

Measure	Interpretation
IQR	Spread of middle 50%; robust to extreme values

2.4 Dataset Profiling

- Row and column count
- Data types
- Missing-value percentage
- Unique values
- Min/max and quantiles
- Category frequencies
- Duplicate count
- Potential invalid values

3. Univariate Analysis

Univariate analysis examines one variable at a time. It is used to understand distribution, frequency, spread, skewness, and potential outliers.

3.1 Histograms

```
import matplotlib.pyplot as plt

plt.hist(df["age"], bins=10)
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.title("Age Distribution")
plt.show()
```

3.2 Frequency Distributions

```
df["department"].value_counts()
df["department"].value_counts(normalize=True)
```

3.3 Outlier Detection and Distribution Interpretation

```
plt.boxplot(df["salary"].dropna())
plt.ylabel("Salary")
plt.title("Salary Box Plot")
plt.show()
```

Interpret the shape: symmetric, right-skewed, left-skewed, multi-modal, or unusually concentrated. Distribution shape can influence preprocessing and model choices.

4. Bivariate Analysis

Bivariate analysis studies the relationship between two variables. The suitable technique depends on whether the variables are numerical or categorical.

4.1 Feature Relationships and Correlation

Correlation measures the strength and direction of a linear relationship between numerical variables.

Correlation does not prove causation, and nonlinear relationships can exist even when correlation is near zero.

```
df.select_dtypes("number").corr()
```

4.2 Scatter Plots

```
plt.scatter(df["advertising"], df["sales"])
plt.xlabel("Advertising Spend")
plt.ylabel("Sales")
plt.title("Advertising vs Sales")
plt.show()
```

4.3 Comparative Analysis

```
df.groupby("department")["salary"].agg(["count", "mean", "median", "min", "max"])
```

4.4 Trend Identification

When one variable is time, sort by time and visualize the metric across periods. Look for trend, seasonality, sudden changes, or missing periods.

5. Multivariate Analysis

5.1 Multiple Feature Relationships

Multivariate analysis examines several variables simultaneously. For example, customer spending may be related to age, income, location, and prior purchase count together rather than to one feature alone.

5.2 Feature Interaction Analysis

An interaction means the effect of one feature can depend on another. For example, the relationship between income and spending may differ across age groups or regions.

5.3 Pattern Discovery and Dimensional Thinking

As the number of variables increases, visual analysis becomes harder. Use grouped summaries, correlation matrices, selected pairwise plots, dimensionality-reduction intuition, and model-informed exploration carefully rather than trying to plot every possible combination.

5.4 Business Insight Generation

A business insight should connect a pattern to a decision. Example: "Customers with 3+ orders have much higher average spend" is more useful when accompanied by counts, effect size, relevant segments, and a check that the pattern is not caused by leakage or sampling bias.

6. Visualization Principles

6.1 Choosing the Right Chart

Question	Recommended Chart
How does a value change over time?	Line chart
How do categories compare?	Bar chart
What is the distribution of a number?	Histogram / box plot
How do two numbers relate?	Scatter plot
How frequent are categories?	Bar chart
Where are potential outliers?	Box plot / scatter plot

6.2 Data Storytelling and Best Practices

- Use a title that states the subject clearly.
- Label axes and units.
- Use readable scales and avoid unnecessary visual effects.

- Show enough context for a fair comparison.
- Keep charts focused on the question.
- Use annotations sparingly for important findings.

6.3 Avoiding Misleading Visuals

- Truncated axes that exaggerate differences
- Unequal intervals presented as equal
- 3D charts that distort area/size
- Cherry-picked time windows
- Missing sample-size context
- Color choices that imply false categories or importance

6.4 Effective Communication

A chart is effective when the intended audience can quickly understand the main comparison, the evidence is faithful to the data, and the conclusion does not go beyond what the analysis supports.

7. ML-Oriented EDA

7.1 Target Variable Analysis

For classification, examine class counts and proportions. For regression, inspect the target distribution, range, skewness, and potential errors. A highly imbalanced classification target can make accuracy misleading.

```
df["target"].value_counts()
df["target"].value_counts(normalize=True)
```

7.2 Feature Importance Awareness

EDA can identify promising features, but simple association is not the same as predictive importance. Consider redundancy, nonlinear effects, interactions, leakage, and whether the feature will exist at inference time.

7.3 Bias Identification

Check whether important groups are adequately represented and whether missingness/error rates differ across groups. Dataset bias can enter through collection, sampling, labeling, or historical processes.

7.4 Data Quality Validation

- Verify missing values and duplicates
- Check impossible ranges
- Inspect category consistency
- Check relationship integrity
- Review outliers
- Inspect target balance/distribution
- Look for leakage clues

7.5 Preparing for Model Building

EDA findings guide preprocessing choices, feature engineering, train/test strategy, evaluation metrics, and model family selection. Record these decisions so modeling remains reproducible.

8. Thinking with AI - EDA

- Ask AI to propose an EDA checklist from column descriptions.
- Ask which charts fit specific variable types and analytical questions.
- Use AI to brainstorm hypotheses, then test them with actual data.

- Verify every generated interpretation against statistics/plots.
- Reject causal claims that are unsupported by the analysis.

Expected Practical Questions - Module 9

1. Perform an initial inspection of a dataset.

Solution / expected approach: Use head, shape, columns, info, describe, missing-value counts, and duplicate counts. State what one row represents.

2. Create a histogram for age and explain what you would look for.

Solution / expected approach: Plot age distribution; inspect center, spread, skewness, gaps, multiple peaks, and implausible values.

3. Display department frequencies and percentages.

Solution / expected approach: Use value_counts() and value_counts(normalize=True)*100.

4. Create a box plot for salary and identify potential outliers.

Solution / expected approach: Use plt.boxplot after handling missing values; investigate rather than automatically deleting flagged points.

5. Calculate a correlation matrix for numeric columns.

Solution / expected approach: Use df.select_dtypes("number").corr(); interpret direction/strength cautiously and remember correlation is not causation.

6. Create a scatter plot of advertising vs sales.

Solution / expected approach: Use plt.scatter; label axes/title and describe trend, spread, clusters, and outliers.

7. Compare average and median salary by department.

Solution / expected approach: Use groupby("department")["salary"].agg(["count", "mean", "median"]).

8. Analyze a classification target for imbalance.

Solution / expected approach: Show counts and percentages. If one class dominates, note that accuracy alone may be insufficient.

9. List at least five checks before model building.

Solution / expected approach: Examples: missingness, duplicates, invalid ranges, leakage, target distribution, feature types, outliers, category consistency.

10. Explain why a strong correlation should not be reported as causation.

Solution / expected approach: Correlation indicates association, not a verified causal mechanism; confounders or reverse relationships may exist.

11. Choose suitable charts for time trend, category comparison, numerical distribution, and numeric relationship.

Solution / expected approach: Line, bar, histogram/box plot, scatter respectively.

12. Write three evidence-based business insights from an EDA.

Solution / expected approach: Each insight should state the observed pattern, numerical/visual evidence, affected segment, and an appropriate next investigation without overclaiming.

Integrated Practical Assessment

The following mini-project connects all four modules. It can be used as a classroom lab, assignment, or practical exam preparation exercise.

Scenario

A company wants to analyze employees and managers, prepare a clean analytical dataset, and produce findings for management. You are given managers and employees tables plus an optional CSV containing employee demographic/attendance information.

Tasks

1. Create normalized managers and employees tables with primary and foreign keys.
2. Insert sample records and demonstrate INSERT, SELECT, UPDATE, and DELETE safely.
3. Write filters for salary ranges and employees with missing manager assignments.
4. Produce salary summaries using COUNT, SUM, AVG, MIN, and MAX.
5. Join employee and manager data and show department for each employee.
6. Find employees earning above the company average using a subquery or CTE.
7. Rank salaries within each department using a window function.
8. Inspect the analytical dataset for missing values, duplicates, invalid salaries, and inconsistent categories.
9. Create at least two derived features (for example monthly_salary or manager_team_size).
10. Prepare categorical/numerical features using appropriate transformations without leakage.
11. Perform univariate, bivariate, and multivariate EDA.
12. Create at least four suitable visualizations and write evidence-based observations.
13. Document all cleaning, transformation, and validation decisions.
14. Summarize how the prepared data could support a future ML model.

Expected Deliverables

- SQL script
- Cleaned dataset or reproducible preprocessing notebook
- EDA notebook/report
- Visualizations with titles/labels
- Short findings summary
- Data-quality and leakage checklist

Practical Viva / Review Questions

- Why is a primary key important?
- What problem does a foreign key solve?
- When would LEFT JOIN be preferred over INNER JOIN?
- What is the difference between WHERE and HAVING?
- Why can a window function be preferable to GROUP BY for ranking?
- Why might an index improve a query but slow some write operations?
- How do 1NF, 2NF, and 3NF reduce redundancy problems?
- Why should missing values not always be filled with the mean?
- What is the difference between normalization and standardization?
- What is data leakage and how can preprocessing create it?
- What is the difference between univariate, bivariate, and multivariate analysis?
- Why does correlation not imply causation?
- How would you detect class imbalance?

- How can a visualization mislead even when the data values are correct?
- What evidence would you collect before removing an outlier?

Quick Revision Checklist

Area	You Should Be Able To
Database Fundamentals	Explain databases, relational structure, keys, relationships, and ML use cases
Core SQL	Create tables, insert/read/update/delete, filter, sort, limit, distinct, aggregate
Advanced SQL	Normalize schemas, join tables, use subqueries/CTEs/window functions, reason about indexes and warehouses
Wrangling	Handle missing/duplicate/invalid/outlier data; transform, encode, scale, merge, aggregate, pipeline
Leakage	Recognize future/target leakage and fit preprocessing correctly
EDA	Profile data, analyze distributions/relationships, select charts, validate quality, communicate insights

Final Study Advice - Do not memorize code without understanding the data question. In practical exams, first state the goal, inspect the data/schema, choose the correct operation, run it, verify the output, and explain why the result is trustworthy.